

Visualization Literacy Analysis

Derek Harter, Shulan Lu

11/25/2025

Read in data files

Raw data is in the ../AccData subdirectory in two excel spreadsheet files per participant. The following code combines into single dataframe of all data.

```
#grab files from google drive (only have to do this once)
# source("getFromGoogleDrive.R")

#get the first 6? characters of each data file
#get unique values of these
# this is list of subject ids

raw_file_names <- list.files("../AccData")
first_six <- substr(raw_file_names, 1, 6)
sub_ids <- unique(first_six)

# fast_RT = data.frame(ParticipantId = character(),
#                        TrialName = character(),
#                        type = character(),
#                        time = numeric()
# )

all_data <- NULL

for (i in 1:length(sub_ids)){

  if (grepl("~$", sub_ids[i], fixed = T)){
    next
  }

  temp_file1 <- read_xlsx(paste0("../AccData/", sub_ids[i], "_1.xlsx")) %>%
    slice(1:17) %>%
    select(-starts_with("Order")) %>%
    rename(correct = 8)

  temp_file2 <- read_xlsx(paste0("../AccData/", sub_ids[i], "_2.xlsx")) %>%
    slice(1:17) %>%
    select(-starts_with("Order")) %>%
    rename(correct = 8)

  new_temp <- temp_file1 %>%
    bind_cols(temp_file2$correct) %>%
```

```

    rename(Correct_1 = correct, Correct_2 = "...9") %>%
    mutate(AnswerRT = TimeToBeginInput - TimeToReadQuestion)

all_data <- all_data %>%
  bind_rows(new_temp)
}

all_data <- all_data %>%
  rename(readRT = TimeToReadQuestion, totalRT = TimeToBeginInput)

#read in trialtype key (I created this from an early version of the previous paper)
trial_type_key <- read.csv("../trial_type_key.csv", stringsAsFactors = F)

all_data <- all_data %>%
  mutate(TrialType = trial_type_key$TrialType[match(TrialName, trial_type_key$TrialName)]) %>%
  mutate(TrialType = paste0("Type", TrialType))

# remove temporary dataframes to keep workspace cleaner as we go forward
rm(new_temp)
rm(temp_file1)
rm(temp_file2)
rm(trial_type_key)

```

Basic checks

Just a sanity check of all the data. Summary of number of subjects in each of the 3 condition types.

```

#how many participants per condition
all_data %>%
  group_by(ParticipantId, Condition) %>%
  summarize(ntrials = n()) %>%
  group_by(Condition) %>%
  summarize(nsubs = n()) %>%
  summary()

```

`summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups` argument.

```

##   Condition      nsubs
## Length:3      Min.   :33.00
## Class :character 1st Qu.:36.00
## Mode  :character Median :39.00
##                Mean   :40.67
##                3rd Qu.:44.50
##                Max.   :50.00

```

#why is the balance so off?

Remove outliers based on RT

Removing outliers, resulting in data frame named `all_data_no_outliers`.

1. All trials where the **AnswerRT** is less than 2 seconds are removed first, which assumes these reactions were too fast so were done without an actual considered response.
2. All trials where the **AnswerRT** is 3 standard deviations different from the mean are removed next.

Table shows summary of mean and standard deviation of answer RT by each of the trial question types, as well as the 3 standard deviation upper and lower bound of the reaction time.

Displayed histogram is a sanity check of the number of trials each participant in all of the data have done. Most participants should do 16 trials. Less than that are from dropped data for outliers.

```
#removing on trial-by-trial basis

#remove answerRTs below 2000ms first
all_data_remove <- all_data %>%
  filter(AnswerRT >= 2000)
dim(all_data)[1]

## [1] 2074
dim(all_data_remove)[1]

## [1] 2067
#drops 7 trials

# also remove AnswerRT that are more than 3 standard deviations away from
# the mean
all_data_no_outliers <- all_data_remove %>%
  group_by(TrialName) %>%
  filter(!(abs(AnswerRT - mean(AnswerRT)) > 3.0*sd(AnswerRT)))
dim(all_data_no_outliers)[1]

## [1] 2020
#drops 47 more trials

rt_data_summary <- all_data_no_outliers %>%
  group_by(TrialName) %>%
  summarize(meanAnswerRT = mean(AnswerRT, na.rm = T),
            sdAnswerRT = sd(AnswerRT, na.rm = T),
            LB = meanAnswerRT - 3*sdAnswerRT,
            UB = meanAnswerRT + 3*sdAnswerRT)
summary(rt_data_summary)

##   TrialName      meanAnswerRT    sdAnswerRT      LB      UB
## Length:17      Min.   : 8963      Min.   : 5156      Min.   :-67692      Min.   : 24429
## Class :character 1st Qu.:25275      1st Qu.:13263      1st Qu.: -35148      1st Qu.: 65715
## Mode  :character Median :29913      Median :19809      Median : -26012      Median : 88817
##              Mean  :33477      Mean  :20886      Mean  : -29182      Mean  : 96137
##              3rd Qu.:44519      3rd Qu.:26920      3rd Qu.: -17508      3rd Qu.:123649
##              Max.  :61772      Max.  :43154      Max.   : -6504      Max.   :191235

rm(rt_data_summary)

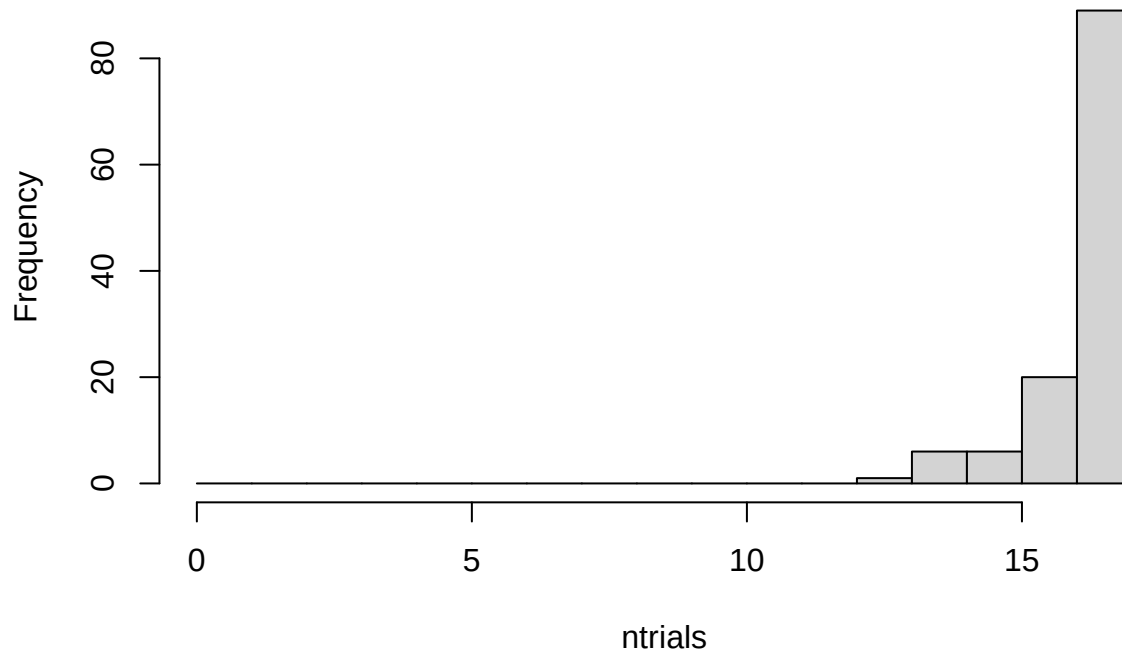
all_data_no_outliers %>%
  group_by(ParticipantId) %>%
```

```

summarize(ntrials = n()) %>%
with(hist(ntrials, breaks = 0:17))

```

Histogram of ntrials



```

rm(all_data_remove)
##maybe consider replacing outliers with means instead of removing them?

```

Extract Only One Type of Chart

Create a dataframe from the data with no outliers that contains only XChartQ1 through XChartQ4 trials for the following analysis.

We replace the `all_data_no_outliers` dataframe with one containing only the selected TrialName question type here, so past this point we are analyzing only this particular TrialName type.

We show the table again of the mean RT for this question type. The table should match the one done with all of the data, but we should only have the particular question trial name type now in the data.

We also again plot the histogram of the number of trials. Participants should have seen 4 of each particular question type, but after removing outliers some only have 3, 2 or 1 remaining.

```

all_data_no_outliers <- all_data_no_outliers %>%
  filter(grepl("SurfacePlotQ[0-9]", TrialName))

rt_data_summary <- all_data_no_outliers %>%
  group_by(TrialName) %>%
  summarize(meanAnswerRT = mean(AnswerRT, na.rm = T),
            sdAnswerRT = sd(AnswerRT, na.rm = T),
            LB = meanAnswerRT - 3*sdAnswerRT,

```

```

UB = meanAnswerRT + 3*sdAnswerRT
summary(rt_data_summary)

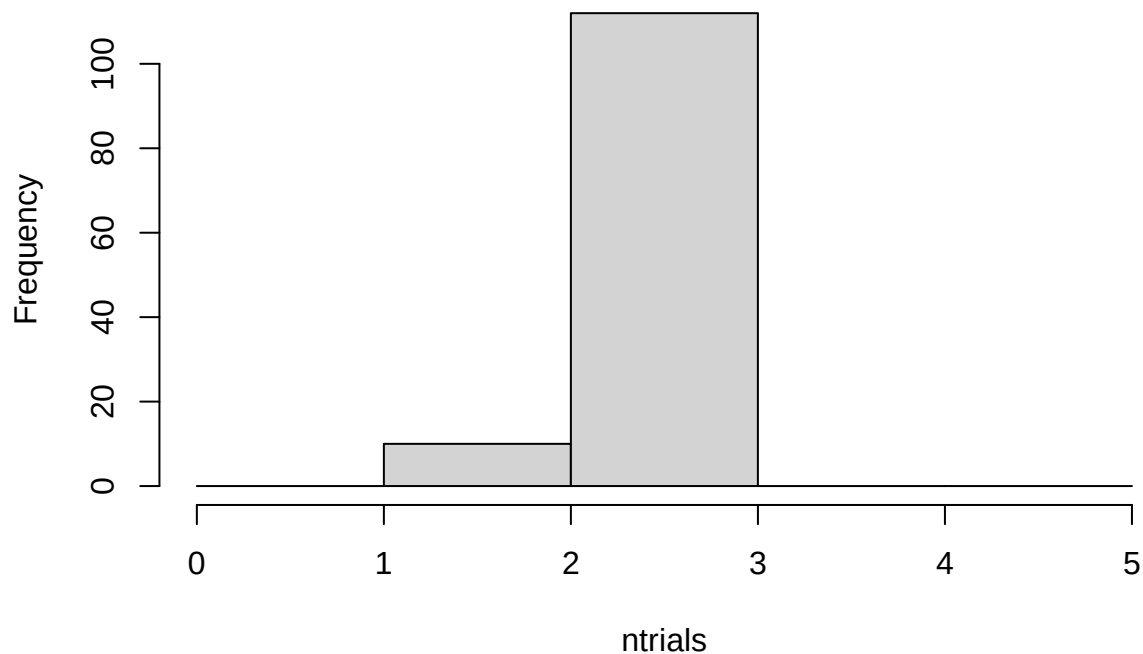
##   TrialName      meanAnswerRT    sdAnswerRT      LB      UB
## Length:3      Min.   :44519    Min.   :29477    Min.   : -67692    Min.   :133778
## Class :character 1st Qu.:48901    1st Qu.:29615    1st Qu.: -56216    1st Qu.:137746
## Mode  :character Median :53283    Median :29753    Median : -44741    Median :141714
##              Mean   :53191    Mean   :34128    Mean   : -49193    Mean   :155576
##              3rd Qu.:57527    3rd Qu.:36454    3rd Qu.: -39944    3rd Qu.:166475
##              Max.   :61772    Max.   :43154    Max.   : -35148    Max.   :191235

rm(rt_data_summary)

all_data_no_outliers %>%
  group_by(ParticipantId) %>%
  summarize(ntrials = n()) %>%
  with(hist(ntrials, breaks = 0:5))

```

Histogram of ntrials



Compare Answer RT

```

#compare answer times

trial_type_means <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition, TrialType) %>%
  summarize(mean_answerRT = mean(AnswerRT), n = n())

```

```
## `summarise()` has grouped output by 'ParticipantId', 'Condition'. You can override using
## the `.groups` argument.
```

```
#rm(trial_type_means)
```

Alternative Analysis using glmm TMB package

The following analysis is for the heterogenous group condition analysis. See .

The following determines if a more complex model is justified.

```
model_glmm_hom = glmmTMB(mean_answerRT ~ Condition * TrialType + (1 | ParticipantId),
                          data=trial_type_means, family = Gamma(link="log"),
                          dispformula = ~ 1, REML = FALSE)
model_glmm_hetero = glmmTMB(mean_answerRT ~ Condition * TrialType + (1 | ParticipantId),
                             data=trial_type_means, family = Gamma(link="log"),
                             dispformula = ~ Condition, REML = FALSE)

anova(model_glmm_hom, model_glmm_hetero)
```

```
## Data: trial_type_means
```

```
## Models:
```

```
## model_glmm_hom: mean_answerRT ~ Condition * TrialType + (1 | ParticipantId), zi=~0, disp=~1
```

```
## model_glmm_hetero: mean_answerRT ~ Condition * TrialType + (1 | ParticipantId), zi=~0, disp=~Condition
```

```
##           Df    AIC    BIC logLik deviance Chisq Chi Df Pr(>Chisq)
```

```
## model_glmm_hom      8 5563.5 5591.4 -2773.8   5547.5
```

```
## model_glmm_hetero  10 5567.0 5601.8 -2773.5   5547.0 0.5898      2    0.7446
```

Try overall effect reporting

```
fixed_effects_anova <- Anova(model_glmm_hom, type="III")
kable(fixed_effects_anova)
```

	Chisq	Df	Pr(>Chisq)
(Intercept)	1.900450e+04	1	0.0000000
Condition	2.756048e+00	2	0.2520761
TrialType	3.387739e-01	1	0.5605378
Condition:TrialType	3.324288e-01	2	0.8468646

```
#summary(fixed_effects_anova, level=0.95)
```

EMM by VR, VR Monitor, VR Monitor Stereo

```
emm_main_condition <- emmeans(model_glmm_hom,
                               specs = ~ Condition,
                               type = "response")
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
summary(emm_main_condition, level=0.95)
```

```
## Condition      response    SE  df asymp.LCL asymp.UCL
## VR              50583 3400 Inf    44331     57717
## VR Monitor      54844 4180 Inf    47237     63676
```

```
## VR Monitor Stereo 44469 3690 Inf 37795 52322
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
```

```
confint(emm_main_condition, level=0.95)
```

```
## Condition response SE df asymp.LCL asymp.UCL
## VR 50583 3400 Inf 44331 57717
## VR Monitor 54844 4180 Inf 47237 63676
## VR Monitor Stereo 44469 3690 Inf 37795 52322
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
```

Preregistered contrasts.

```
comparison_results <- contrast(
  emm_main_condition,
  method = list(
    VR_vs_Monitors_Collapsed = c('VR' = 1, 'VR Monitor' = -0.5, 'VR Monitor Stereo' = -0.5)
  )
)
```

```
summary(comparison_results, infer = TRUE)
```

```
## contrast ratio SE df asymp.LCL asymp.UCL null z.ratio p.value
## VR_vs_Monitors_Collapsed 1.02 0.0897 Inf 0.863 1.22 1 0.274 0.7843
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
## Tests are performed on the log scale
```

```
confint(comparison_results, level = 0.95)
```

```
## contrast ratio SE df asymp.LCL asymp.UCL
## VR_vs_Monitors_Collapsed 1.02 0.0897 Inf 0.863 1.22
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
```

Preregistered contrasts.

```
comparison_results <- contrast(
  emm_main_condition,
  method = list(
    VRMonitor_vs_others_collapsed = c('VR' = -0.5, 'VR Monitor' = 1, 'VR Monitor Stereo' = -0.5)
  )
)
```

```
summary(comparison_results, infer = TRUE)
```

```
## contrast ratio SE df asymp.LCL asymp.UCL null z.ratio p.value
## VRMonitor_vs_others_collapsed 1.16 0.107 Inf 0.964 1.39 1 1.564 0.1177
```

```
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
## Tests are performed on the log scale
confint(comparison_results, level = 0.95)

## contrast          ratio    SE df asymp.LCL asymp.UCL
## VRMonitor_vs_others_collapsed  1.16 0.107 Inf    0.964    1.39
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
```

EMM on Condition and Interaction

If the glmm analysis is not significant, just use the homogeneous model. Otherwise report on the more complex heterogeneous model?

```
emm_main_condition <- emmeans(model_glmm_hom,
                              specs = ~ Condition,
                              type = "response")

## NOTE: Results may be misleading due to involvement in interactions

condition_results <- pairs(emm_main_condition)

summary(condition_results, level=0.95)

## contrast          ratio    SE df null z.ratio p.value
## VR / VR Monitor      0.922 0.0936 Inf    1  -0.797  0.7046
## VR / VR Monitor Stereo  1.137 0.1210 Inf    1   1.209  0.4479
## VR Monitor / VR Monitor Stereo 1.233 0.1390 Inf    1   1.865  0.1489
##
## Results are averaged over the levels of: TrialType
## P value adjustment: tukey method for comparing a family of 3 estimates
## Tests are performed on the log scale
confint(condition_results, level=0.95)

## contrast          ratio    SE df asymp.LCL asymp.UCL
## VR / VR Monitor      0.922 0.0936 Inf    0.727    1.17
## VR / VR Monitor Stereo  1.137 0.1210 Inf    0.886    1.46
## VR Monitor / VR Monitor Stereo 1.233 0.1390 Inf    0.948    1.61
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Conf-level adjustment: tukey method for comparing a family of 3 estimates
## Intervals are back-transformed from the log scale
#kable(condition_results)

emm_interaction <- emmeans(model_glmm_hom,
                           specs = ~ Condition * TrialType,
                           type = "response")
```



```

simple_effects_results <- pairs(emm_interaction, by = "TrialType")

summary(simple_effects_results, level=0.95)

## TrialType = Type1:
## contrast ratio SE df null z.ratio p.value
## VR / VR Monitor 0.892 0.106 Inf 1 -0.962 0.6010
## VR / VR Monitor Stereo 1.109 0.138 Inf 1 0.831 0.6837
## VR Monitor / VR Monitor Stereo 1.243 0.164 Inf 1 1.654 0.2232
##
## TrialType = Type2:
## contrast ratio SE df null z.ratio p.value
## VR / VR Monitor 0.953 0.112 Inf 1 -0.404 0.9138
## VR / VR Monitor Stereo 1.166 0.144 Inf 1 1.246 0.4260
## VR Monitor / VR Monitor Stereo 1.223 0.159 Inf 1 1.550 0.2676
##
## P value adjustment: tukey method for comparing a family of 3 estimates
## Tests are performed on the log scale

confint(simple_effects_results, level=0.95)

## TrialType = Type1:
## contrast ratio SE df asymp.LCL asymp.UCL
## VR / VR Monitor 0.892 0.106 Inf 0.676 1.18
## VR / VR Monitor Stereo 1.109 0.138 Inf 0.828 1.49
## VR Monitor / VR Monitor Stereo 1.243 0.164 Inf 0.913 1.69
##
## TrialType = Type2:
## contrast ratio SE df asymp.LCL asymp.UCL
## VR / VR Monitor 0.953 0.112 Inf 0.723 1.26
## VR / VR Monitor Stereo 1.166 0.144 Inf 0.873 1.56
## VR Monitor / VR Monitor Stereo 1.223 0.159 Inf 0.902 1.66
##
## Confidence level used: 0.95
## Conf-level adjustment: tukey method for comparing a family of 3 estimates
## Intervals are back-transformed from the log scale

#kable(simple_effects_results)

```

Compare Accuracy

Extract wide dataframe

Do all of the previous again but on the Accuracy results.

We again create a dataframe in needed format of only the accuracy data.

A bit different from before, only need to create an average of the two correct features we have. But then we do a similar transformation into wide format and replace na/missing values.

```

all_data_no_outliers <- all_data_no_outliers %>%
  rowwise() %>%
  mutate(Correct_Avg = mean(c(Correct_1, Correct_2)))

trial_type_mean_accuracy <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition, TrialType) %>%

```

```

summarize(mean_acc = mean(Correct_Avg),
           n = n())

## `summarise()` has grouped output by 'ParticipantId', 'Condition'. You can override using
## the `.groups` argument.

table(all_data_no_outliers$Correct_Avg)

##
##      0 0.25  0.5 0.75    1
## 179   20   52  10   95

#rm(trial_type_mean_accuracy)

# Step 1: Aggregate to the Participant Level
# In a mixed design, the experimental unit is the Participant.
# We first average all raw trials for each person within each Trial Type.
subject_means <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition, TrialType) %>%
  summarize(
    subject_avg = mean(Correct_Avg, na.rm = TRUE),
    .groups = "drop"
  )

# Step 2: Compute Descriptive Stats for the Groups
# Now we calculate the stats across participants.
descriptive_stats <- subject_means %>%
  group_by(Condition) %>%
  summarize(
    n = n(),                                # Number of subjects per group
    Mean = mean(subject_avg),               # Grand Mean
    SD = sd(subject_avg),                   # Standard Deviation (Variability)
    SE = SD / sqrt(n),                      # Standard Error

    # 95% Confidence Interval Calculation
    # We use qt(0.975) for the two-tailed t-distribution critical value
    CI_Margin = qt(0.975, df = n - 1) * SE,
    CI_Lower = Mean - CI_Margin,
    CI_Upper = Mean + CI_Margin,
    .groups = "drop"
  )

# View the final table
print(descriptive_stats)

## # A tibble: 3 x 8
##   Condition          n Mean    SD    SE CI_Margin CI_Lower CI_Upper
##   <chr>          <int> <dbl> <dbl> <dbl>    <dbl>    <dbl>    <dbl>
## 1 VR            99 0.415 0.405 0.0407  0.0808    0.335    0.496
## 2 VR Monitor    77 0.354 0.382 0.0436  0.0868    0.267    0.441
## 3 VR Monitor Stereo 65 0.435 0.396 0.0491  0.0981    0.336    0.533

# Step 2: Compute Descriptive Stats for the Groups
# Now we calculate the stats across participants.
descriptive_stats <- subject_means %>%
  group_by(TrialType) %>%

```

```

summarize(
  n = n(),                                # Number of subjects per group
  Mean = mean(subject_avg),               # Grand Mean
  SD = sd(subject_avg),                   # Standard Deviation (Variability)
  SE = SD / sqrt(n),                      # Standard Error

  # 95% Confidence Interval Calculation
  # We use qt(0.975) for the two-tailed t-distribution critical value
  CI_Margin = qt(0.975, df = n - 1) * SE,
  CI_Lower = Mean - CI_Margin,
  CI_Upper = Mean + CI_Margin,
  .groups = "drop"
)

# View the final table
print(descriptive_stats)

```

```

## # A tibble: 2 x 8
##   TrialType      n Mean    SD    SE CI_Margin CI_Lower CI_Upper
##   <chr>    <int> <dbl> <dbl> <dbl>    <dbl>    <dbl>    <dbl>
## 1 Type1      119 0.502 0.417 0.0383  0.0758    0.426    0.578
## 2 Type2      122 0.302 0.346 0.0314  0.0621    0.240    0.364

```

Binary Collapse

```

library(ordinal)

##
## Attaching package: 'ordinal'

## The following object is masked from 'package:dplyr':
##
##   slice

# Collapse to Binary: Incorrect (0) vs. Correct (1)
# Treat 0.25, 0.5, 0.75 as "partially correct" = Correct
all_data_no_outliers <- all_data_no_outliers %>%
  mutate(Correct_Binary = ifelse(Correct_Avg == 0, 0, 1),
         Correct_Binary_Factor = factor(Correct_Binary,
                                       levels = c(0, 1),
                                       labels = c("Incorrect", "Correct"),
                                       ordered = TRUE))

# Check distribution
table(all_data_no_outliers$Correct_Binary_Factor)

##
## Incorrect   Correct
##      179      177

table(all_data_no_outliers$Correct_Binary_Factor,
      all_data_no_outliers$Condition)

##
##           VR VR Monitor VR Monitor Stereo
## Incorrect 70          63          46

```

```
## Correct 76 50 51
# Fit binary ordinal model (essentially logistic regression with random effects)
model_clmm_binary <- clmm(Correct_Binary_Factor ~ Condition * TrialType + (1 | ParticipantId),
  data = all_data_no_outliers,
  link = "logit")

summary(model_clmm_binary)

## Cumulative Link Mixed Model fitted with the Laplace approximation
##
## formula: Correct_Binary_Factor ~ Condition * TrialType + (1 | ParticipantId)
## data: all_data_no_outliers
##
## link threshold nobis logLik AIC niter max.grad cond.H
## logit flexible 356 -223.49 460.98 308(918) 8.98e-05 8.0e+01
##
## Random effects:
## Groups Name Variance Std.Dev.
## ParticipantId (Intercept) 1.67 1.292
## Number of groups: ParticipantId 122
##
## Coefficients:
## Estimate Std. Error z value Pr(>|z|)
## ConditionVR Monitor -0.07314 0.59710 -0.122 0.9025
## ConditionVR Monitor Stereo 0.82986 0.66988 1.239 0.2154
## TrialTypeType2 -1.08236 0.42835 -2.527 0.0115 *
## ConditionVR Monitor:TrialTypeType2 -0.57953 0.65492 -0.885 0.3762
## ConditionVR Monitor Stereo:TrialTypeType2 -1.16637 0.72243 -1.615 0.1064
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Threshold coefficients:
## Estimate Std. Error z value
## Incorrect|Correct -0.8369 0.3974 -2.106
```